Generative AI Tools Usage

Google Gemini

Prompt 1

Question

```
Hello. I started a research on the topic 'A Longitudinal Study On Programming Education Using Chatbots'.
The study will focus on developing and evaluating an LLM-powered programming assistant for helping students solve programming exercises by providing hints instead of direct solutions to their
problems. The primary goal is to aid learning by guiding students towards solutions, preventing over-reliance on direct answers.

I attached the references of my study (myBib.bib), all 28 papers I want to group the 28 papers in these five categories I defined as part of my SOTA chapter.

Foundations of Programming Assistance: From Human UTAs to Chatbots

LLM-Powered Tutoring Systems: Design, Guardrails, and Hinting Philosophies

Advanced Scaffolding: Socratic Interaction and Metacognition

Technical Implementation: Local Models, Privacy, and Performance

The Longitudinal Impact: Metacognition, Trust, and Learning Outcomes
```

Output

```
To help you structure your State of the Art (SOTA) chapter, I have categorized all 28 papers into your five specified LaTeX sections. Each entry includes a brief justification for its placement.

---

### Foundations of Programming Assistance: From Human UTAs to Chatbots

*This section establishes the historical and pedagogical context of student support, from the human role to the initial shift toward automated visualization and AI.*

1. **Mirza et al. (2019):** A systematic review of Undergraduate Teaching Assistants (UTAs) that provides the baseline for human peer support.
2. **Riese and Kann (2020):** Explores the multi-faceted roles and conflicts of human TAs, highlighting the gap that chatbots can fill.
3. **Guo (2013):** The "Online Python Tutor" paper, representing the pre-LLM era of program visualization and step-by-step execution help.
4. **Becker et al. (2023):** A position paper on the "hard" transition from traditional programming to AI-driven generation, setting the stage for the rest of the chapter.
5. **Raihan et al. (2025):** A systematic literature review that provides a high-level map of how LLMs have entered the CS education space.

### LLM-Powered Tutoring Systems: Design, Guardrails, and Hinting Philosophies

*This section covers the architecture and "rules" of modern assistants designed specifically to avoid giving direct answers.*

6. **Kazemitabaar et al. (2024) [CodeAid]:** Establishes the core philosophy of providing conceptual feedback and pseudo-code without direct code solutions.
7. **Liffiton et al. (2024) [CodeHelp]:** Focuses on the implementation of "guardrails" to ensure AI remains a tutor and not a "solution-generator."
8. **Iris Ma et al. (2024) [RAGMan]:** Details the use of Retrieval Augmented Generation (RAG) to keep AI tutors grounded in course-specific materials.
9. **Sebastian Zhao et al. (2025) [CodingGenie]:** Introduces a proactive assistant integrated into the editor to provide bug-fixing suggestions autonomously.
10. **Valerie Chen et al. (2025):** Investigates the design elements of proactive assistants and how they impact programmer productivity.
11. **Kimmel et al. (2024):** Examines the integration of AI into assessment tools (Athene) to explain compiler and logic errors in real-time.
12. **Nelson et al. (2025) [SENSAI]:** Shows how assistants can extract "working context" (active files/terminals) to provide personalized, cost-effective guidance.
13. **Molina et al. (2025):** Focuses on design considerations for non-native English speakers, leveraging LLM's multilingual capabilities.

### Advanced Scaffolding: Socratic Interaction and Metacognition

*This section deals with the "how" of interaction—Socratic questioning, decomposition, and progressive hint ladders.*

14. **Jiayu Liu et al. (2024) [SocraticLM]:** Proposes the "Thought-Provoking" paradigm, training models to act like classroom teachers rather than Q&A bots.
15. **Xiao et al. (2024) [Hint Factory]:** Explores the granularity of hints, showing how multiple levels (from natural language to commented examples) support learners.
16. **Shuai Ma et al. (2025) [DBox]:** Scaffolds learning through "co-decomposition," helping students break down algorithmic problems into a "step tree."
17. **Boxuan Ma et al. (2025):** Analyzes student-AI dialogue logs through a metacognitive lens (planning, monitoring, evaluating).
18. **Heickal and Lan (2024):** Introduces the "Feedback-Ladder," a structured approach to layered hints for logical errors.
19. **Birillo et al. (2024):** Combines LLMs with static analysis to generate "next-step hints" that are both pedagogically and technically sound.
20. **Roest et al. (2024) [StAP-tutor]:** Investigates prompt engineering practices specifically for introductory Python next-step hints.
21. **Hou et al. (2024) [CodeTailor]:** Transforms errors into "Parsons Puzzles," forcing students to reassemble logic rather than just reading a solution.

### Technical Implementation: Local Models, Privacy, and Performance

*This section is vital for your project's technical goals: local Ollama deployment, open-source performance, and privacy.*

22. **Reza et al. (2025):** The "Small Models, Big Support" framework for locally deployable (3B-7B) models using RAG and CAG.
23. **Kotalwar et al. (2024):** Benchmarks in-browser inference (Llama-3-8B, Phi-3) specifically for programming feedback generation.
24. **Koutcheme et al. (2025):** A comparative study showing that open-source models are nearly on par with proprietary ones (like GPT-4) for judging feedback quality.

### The Longitudinal Impact: Metacognition, Trust, and Learning Outcomes

*This section reviews field studies and long-term research on how these tools affect learning over time and the risks involved.*

25. **Prather et al. (2024):** Highlights the "widening gap" where struggling novices may experience an "illusion of competence" when using AI.
26. **Wenhan Lyu et al. (2024) [CodeTutor]:** A semester-long field study showing performance gains but also a shift in student preference back to human TAs over time.
27. **Chang Liu et al. (2025):** A large-scale investigation into 24/7 interaction patterns across multiple computer science courses.
28. **Peters and Scharlau (2025):** Explores the psychology of interacting with fallible AI and how "distrust" affects human-AI task performance.

This categorization gives you a clear narrative: you start with the **history (S1)**, explain the **current designs (S2)** and **teaching strategies (S3)**, justify the **technical choices (S4)**,
and conclude with the **long-term effects (S5)** you intend to measure.
```

Prompt 2

Question

```
can you tell me a bit about https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct? I am curious about choosing it for some experiments in google colab
```

Output

```
The model hosted at `meta-llama/Meta-Llama-3-8B-Instruct` is one of the most significant "open-weights" large language models released by Meta (Facebook).

It is likely the specific engine you used for your **Socratic AI Teaching Assistant** project, as it strikes the current "sweet spot" between reasoning capability and the ability to run on free or
consumer-grade hardware (like Google Colab's T4 GPU).

Here is a breakdown of what this model is and why it matters:

### 1. What is it?

* **Developer:** Meta.
* **Release Date:** April 18, 2024.
* **Type:** It is an **"Instruction Tuned"** model. This means that unlike the "Base" version (which just completes text), this version has undergone Supervised Fine-Tuning (SFT) and Reinforcement
Learning (RLHF) specifically to follow user commands, answer questions, and act as an assistant.
* **Size:** **8 Billion parameters**. This is considered "small" in the world of LLMs (compared to GPT-4's estimated trillions), but it is highly efficient.

### 2. Why is it famous? (The "Punching Above Its Weight" Factor)

Before Llama 3, models of this size (7B-8B) were usually considered "dumb" for complex logic or coding. Llama 3 changed that.
```

```
* **Massive Training Data:** It was trained on over **15 Trillion tokens** of text. For context, most models its size are trained on only 2-3 trillion. This "over-training" makes it incredibly dense with knowledge despite its small file size.
* **Code Capabilities:** It scores very high on coding benchmarks (HumanEval), which explains why it was a good choice for your Python tutor project.
* **Reasoning:** It outperforms much larger models (like Llama 2 70B) on reasoning tests, meaning it can handle the logic required to be a "Socratic" guide rather than just a text generator.

### 3. Key Technical Specs

* **Context Window:** **8,192 tokens** (approx. 6,000 words). This determines how much conversation history (or how many PDF pages) it can remember at once.
* **Architecture:** It uses **Grouped-Query Attention (GQA)**, a technique that speeds up inference, making it feel very snappy when you chat with it.
* **Knowledge Cutoff:** March 2023 (for the 8B version).

### 4. Why it might be a good fit for you?

* **Local Execution:** Because it is only 8B parameters, it can be "quantized" (compressed) to 4-bit mode (using the `unsloth` or `bitsandbytes` libraries). This allows it to fit into ~6GB of VRAM, making your project accessible without expensive servers.
* **Privacy:** Since it runs locally (via tools like Ollama), you don't need to send the data to OpenAI.
```

Prompt 3

Question

```
how can I set a custom color to a table header in latex
```

Output

````
To set a custom color for a table header in LaTeX, you will need the **`xcolor`** package with the `table` option.

Here is the step-by-step breakdown and a complete code example.

### 1. The Preamble Setup

First, add the package to your preamble and define your custom color using `\definecolor`. You can define colors using HTML hex codes, RGB, or cmyk.

```latex
\usepackage[table]{xcolor} % Enables table coloring

% Define your custom color (e.g., a nice navy blue using Hex)
\definecolor{CustomNavy}{HTML}{1F4E79}
```

### 2. Apply it to the Table Header

Inside your `tabular` environment, place the **`\rowcolor{...}`** command at the very beginning of your header row.

```latex
\documentclass{article}
\usepackage[table]{xcolor}

% Define custom colors
\definecolor{CustomNavy}{HTML}{1F4E79}
\definecolor{WhiteText}{HTML}{FFFFFF}

\begin{document}

\begin{table}[h]
\centering
\begin{tabular}{|c|c|c|}
    \hline
    % Apply row color and make text white/bold for contrast
    \rowcolor{CustomNavy}
    \color{WhiteText}\textbf{Column 1} & \color{WhiteText}\textbf{Column 2} & \color{WhiteText}\textbf{Column 3} \\ \hline
    Item A & Item B & Item C \\ \hline
    Item D & Item E & Item F \\ \hline
\end{tabular}
\caption{Table with a custom colored header.}
\end{table}

\end{document}
```

### Quick Tips for Clean Tables

* **Text Contrast:** If you use a dark background color for the header, remember to change the text color to white using `\color{white}` or a defined light color so it remains readable.
* **Line Clipping Fix:** Sometimes horizontal grid lines (`\hline`) right next to colored rows can look thin or disappear in PDF viewers. If you run into this, consider using the `booktabs` package for professional, line-free table styling, or use `\arrayrulecolor` to match your grid lines to your theme.
````

Prompt 4

Question

```
I just bought gemini api key credits, of 15\$ and was wondering if that money can be spent only on a api key on a designated project, or if I can create a new api key/new project + api key and use the same credits there. or is the money project specific?
```

Output

```
It is an exciting step to start building with the Gemini API! To give you a straightforward answer: **your $15 is not locked to a single project or API key.** Here is exactly how the billing architecture works so you can manage your funds effectively:

### **Credits are Tied to the Billing Account**

When you prepay for Gemini API credits, that money goes into a central **Billing Account**, rather than being deposited into a specific project. Because of this top-down structure, your funds are highly flexible.

### **Sharing Across Multiple Projects and Keys**

You can absolutely create a new project, generate a new API key, and use those same credits. As long as you link any new or existing projects to the **same billing account** where you added the $15, they will all share that exact same pool of money.

* **Multiple Keys in One Project:** All keys will draw from the $15.
* **Multiple Projects with Different Keys:** As long as the projects share the billing account, they will all draw from the same $15.

---

### **A Quick Word of Caution**

Because all linked projects pull from the same prepaid balance, if one of your projects heavily consumes the API and hits the $0 mark, **all API keys across all linked projects will stop working simultaneously.** **How to protect your credits:**
If you plan on creating multiple projects (for example, one for testing and one for production), you can set **project-level spend caps** within Google AI Studio. This allows you to tell the system, *"Only let Project A use $5 of my total $15,"* ensuring that an experimental script doesn't accidentally drain the funds meant for your main project.
```

Prompt 5

Question

```
is this sentence making sense or should it be reversed? we can provide a safety net for students who encourage "active learning" rather than dependency on a system
```

Output

```
It needs a slight adjustment. As it is currently written, the sentence implies that the **students** are the ones doing the encouraging.

What you likely mean is that the **safety net** (or your program) is what encourages active learning.

Here is why it feels off: By using "who encourage," the action gets tied to the students. By changing it to "that encourages," the action gets tied to the safety net.

Here are a few ways to rewrite it so your meaning is crystal clear:

**Option 1: The easiest fix (just changing two words)**

> "We can provide a safety net for students **that encourages** 'active learning' rather than dependency on a system."

**Option 2: A bit more active and flowing**

> "We can provide a safety net that encourages students to engage in 'active learning' rather than depending on a system."

**Option 3: Punchy and professional**

> "We can provide students with a safety net that promotes 'active learning' instead of system dependency."

If you want to emphasize the contrast the most, **Option 2** is generally the strongest choice because it clearly places the students as the ones *doing* the active learning.
```

Prompt 6

Question

```
how can I use tcolorbox for this paragraph, where the Indirect answer is the title and the rest the content

Indirect answer: The literature provides strong evidence that guardrailed, hint-based approaches improve learning outcomes compared to unrestricted code generation, though the effect is modulated by
student skill level and hint design quality.
```

Output

```
Here is how you can wrap that paragraph using the `tcolorbox` package.

First, make sure you have loaded the package in your document's **preamble**:

```latex
\usepackage{tcolorbox}

```

Then, you can use the following code for the paragraph:

```latex
\begin{tcolorbox}[title={Indirect answer:}]
    The literature provides strong evidence that guardrailed, hint-based approaches
    improve learning outcomes compared to unrestricted code generation, though the
    effect is modulated by student skill level and hint design quality.
\end{tcolorbox}

```

Optional: Styled Version

If you want a more modern, clean look (sharp corners, subtle colors, and no bold white-on-black default header), you can customize it like this:

```latex
\begin{tcolorbox}[
    colback=gray!5,     % Light gray background
    colframe=blue!70!black, % Deep blue border and title background
    fonttitle=\bfseries,    % Bold title
    title={Indirect answer:}
]
    The literature provides strong evidence that guardrailed, hint-based approaches
    improve learning outcomes compared to unrestricted code generation, though the
    effect is modulated by student skill level and hint design quality.
\end{tcolorbox}

```
```

Claude

Prompt 7

Question

```
So I have a latex overleaf project, that used to work perfectly fine, I tried making a copy and renaming that and now it does not work anymore, not the copy not the initial project.
I keep getting Your compile timed out
Your project exceeded the compile timeout limit on our free plan.
but for similarly big projects it works and don't understand what happened, I tried so many things and still doesn't work

the only thing that kinda worked was to move the end document command above the bibliography section, it loads the document, but well no bibliography. what else should I try to do?
```

Output

```
The bibliography is almost certainly the culprit. Here are things to try, roughly in order of likelihood:

**Check for a corrupted or bloated .bbl file.** If you're using BibTeX/BibLaTeX, sometimes the auxiliary files get into a bad state — especially after copying/renaming a project. In Overleaf, go to Logs and output files → click the trash icon to clear the cache, then recompile. This is the most common fix after renaming a project.

**Trim your .bib file.** If your .bib file contains hundreds of entries but you only cite a fraction of them, the compiler still has to parse everything. Create a slimmed-down version with only the entries you actually `\cite{}`. Even a few dozen unused entries with malformed fields can slow things dramatically.

**Look for a bad .bib entry.** A single malformed entry (unclosed brace, weird Unicode, a field with an enormous abstract or embedded LaTeX) can send BibTeX into a spiral. Try binary-searching: comment out half your `\cite` commands and half the .bib entries, see if it compiles, then narrow down.

**Switch compilation approach.** If you're using `bibtex`, try switching to `biber` (or vice versa) in Overleaf's menu settings. Sometimes one handles edge cases better than the other. Also make sure your Menu → Compiler setting (pdfLaTeX vs XeLaTeX vs LuaLaTeX) matches what the project actually needs — XeLaTeX and LuaLaTeX are significantly slower.

**Pre-compile the bibliography.** As a workaround, compile once with bibliography included (even if it times out, it might generate a `.bbl` file), then download that `.bbl` file, include it directly with `\input{filename.bbl}` instead of `\bibliography{...}`, and comment out the `\bibliography` and `\bibliographystyle` commands. This skips the BibTeX step entirely on subsequent compiles.

**Check for circular or recursive inputs.** When you copied and renamed the project, you might have accidentally introduced a file that `\input`s or `\include`s itself, or a reference to the old project structure. Look through your `.tex` files for any stale paths.
```

```
If none of that works, you could share the structure of your preamble and bibliography setup and I can look for specific issues.
```

Prompt 8

Question

```
do we need these Kruskal–Wallis H values anymore?
```

Output

```
The Kruskal–Wallis H test was meant to answer: "Is the quality difference between the 5 models statistically significant, or could it be due to chance?" That's a valid question for your RQ5 (cloud
vs local comparison).

But honestly, looking at your data, the answer is obvious from the averages alone — cloud models score 7.15, local models score 5.01, that's a 30% gap visible to anyone reading the table. With n=80
responses per model and such a clear separation, a formal statistical test adds rigour but doesn't change the conclusion. The cloud-only comparison (Gemini vs Claude vs GPT) is also clearly non-
significant — they're within 0.2 points of each other.

So you can keep it simple: State the averages and the gap percentage, say the cloud models performed comparably to each other while local models scored ~30% lower, and leave it at that. The reader
can see it.

Given that your participant pool is n=10 experienced developers (not the target audience), and the chapter already acknowledges this as a limitation, I'd lean toward option 1 — the descriptive
statistics tell the story clearly enough.
```

Prompt 9

Question

```
I found an anomaly for the parsons puzzle. for this problem the puzzle contains many comments. and does not make sense to rearrange these.
       {
          "response_text": "That usually means there’s an edge case your code isn’t handling.\n\nCommon anagram test-case failures:\n\n- **Upper/lowercase**\n  - `\"Listen\"` vs `\"silent\"`\n-
**Spaces**\n  - `\"rail safety\"` vs `\"fairy tales\"`\n- **Punctuation**\n  - `\"A gentleman!\"` vs `\"Elegant man\"`\n- **Repeated letters**\n  - `\"aabb\"` vs `\"ab\"`\n- **Different lengths
after cleanup**\n- **Unicode/accents** depending on the problem\n\n### If you’re using sorting, make sure you normalize first\n```python\ndef is_anagram(s1, s2):\n    s1 = ''.join(ch.lower() for ch
in s1 if ch.isalnum())\n    s2 = ''.join(ch.lower() for ch in s2 if ch.isalnum())\n    return sorted(s1) == sorted(s2)\n```\n\n### If the problem expects only letters and exact characters\nThen you
may need something simpler:\n```python\ndef is_anagram(s1, s2):\n    return sorted(s1) == sorted(s2)\n```\n\n### Best next step\nPaste:\n1. your code, and\n2. one failing test case\n\nand I’ll point
out exactly why it fails.",
          "mode": "socratic",
          "code_police_triggered": false,
          "message_id": "39398ed6-da0e-4e0d-a838-bbc88897d7e3",
          "challenge_solved": false,
          "sources": null,
          "latency_ms": 3880.6,
          "analysis": {
            "type": "analysis",
            "language": "python",
            "is_valid": true,
            "feedback": "Syntax is valid Python.",
            "errors": []
          },
          "is_puzzle": false,
          "puzzle_lines": null
        },
        {
          "response_text": "You seem stuck, so let's try a different approach! Instead of writing code, can you arrange these scrambled lines in the correct order?",
          "mode": "puzzle",
          "code_police_triggered": false,
          "message_id": "9e5bd4ed-3958-47c9-8635-d1e072e0ff78",
          "challenge_solved": false,
          "sources": null,
          "latency_ms": 1604.5,
          "analysis": {
            "type": "analysis",
            "language": "python",
            "is_valid": true,
            "feedback": "Syntax is valid Python.",
            "errors": []
          },
          "is_puzzle": true,
          "puzzle_lines": [
            "         # fastest way till now with reference to the others' submissions",
            "         :rtype: bool",
            "         if s == t:",
            "         #     if i not in dic:",
            "         # for i in t:",
            "class Solution:",
            "                 return False",
            "         :type s: str",
            "         \"\"\"",
            "         # dic = {}",
            "         #         return False",
            "         for i in map(chr, range(97, 123)):",
            "         #         dic[item] = 1",
            "     def isAnagram(self, s, t):",
            "         # return all(value == 0 for value in dic.values())",
            "             if s.count(i) != t.count(i):",
            "         if len(s) != len(t):",
            "             return False",
            "         # for item in s:",
            "         #     else:",
            "         :type t: str",
            "         #         dic[i] -= 1",
            "         #     if item not in dic:",
            "         #         dic[item] += 1",
            "             return True",
            "         return True",
            "         #     else:",
            "         \"\"\""
          ]
        }
      ]
    },
```

Output

```
The APPS dataset solution for exercise #2420 includes commented-out alternative approaches. generate_puzzle() just shuffles all lines, including comments. The puzzle becomes nonsensical.

Fix: filter out comment-only lines and blank lines before generating the puzzle.

All three need code fixes + re-running the affected experiments. Want me to fix all three now? Here's what I'd do:

1. Backend fix in tutor.py: append code to user message when enable_socratic_prompt=false
2. Backend fix in parsons.py: filter comment lines from puzzle generation
3. Update test_cases.json: change RAG questions to match "Learning Python" content
4. You upload the PDF locally through localhost:4200
5. Re-run the experiment script (checkpoint will retry only the changed test cases — or we can clear the checkpoint to re-run everything)
```

Question

```
but if we filter the comments, will the program validate that the code is rearranged in the correct order?
```

Output

```
Good question — let me check exactly how the puzzle validation works.

Yes — it's a direct comparison: student_order == correct_order. Both lists come from the same filtering. So if we filter out comments in generate_puzzle(), both correct_order and shuffled_lines will
exclude them, and check_puzzle_solution() will validate against the filtered set. It stays consistent.

The fix is just changing one line:

# Before: keeps all non-empty lines (including comments)
lines = [line for line in correct_code.split("\n") if line.strip()]

# After: also filters out comment-only lines
lines = [line for line in correct_code.split("\n") if line.strip() and not line.strip().startswith("#")]

The student rearranges only the meaningful code lines, and the validation checks against those same meaningful lines. Want me to go ahead and fix all three issues?
```

Question

```
does the fix for parsons puzzle cover these types of lines? :type s: str and  \"\"\" (shown as """ in the puzzle)?
```

Output

```
No — the current fix only filters # comments. Those docstring lines (:type s: str, """) would still get through.

The cleanest approach: filter out everything that isn't executable code — comments, docstring delimiters, and docstring content (:type, :rtype, :param).
```

Prompt 10

Question

```
I noticed that some messages were cut off mid sentance in the initial tests (e.g. "response_text": "A `for` loop in Python is used to **iterate** over a sequence (like a list, tuple, string, or
range) or other iterable objects. It allows you to execute") why?
```

Output

```
That's the cause — max_tokens=1024 is the default. The tutor calls provider.generate(messages=messages, temperature=0.6) without specifying max_tokens, so it caps at 1024 tokens. For verbose
responses, the model hits the limit and gets cut off mid-sentence.
```

Question

```
but why only some were truncated and some where not?
```

Output

```
Because some responses are shorter than 1024 tokens and some are longer. It depends on how verbose the model is for that particular question.

- Gemini tends to be more verbose → hits the 1024 cap more often → truncated
- Shorter answers (simple questions, models that follow the "3-5 sentences" instruction) → finish within 1024 → complete
- C0 (no guardrails) → no system prompt telling the model to be concise → model rambles → more likely to truncate
- C1-C3 (with Socratic prompt) → system prompt says "Keep answers focused but thorough (3-5 sentences)" → usually fits within 1024

So the truncation is random depending on model verbosity for each specific question. It's not a bug — just a low default.
```

Prompt 11

Question

```
Can you perform a security audit on the entire fe project, looking for any issues, bugs, vulnerabilities, faulty implementations, anything that can bring risks to our application. Be as thorough as
possible, looking at every file.
```

Output

```
Here's the synthesized security audit across all 93 frontend files.

---

Frontend Security Audit

CRITICAL

┌─────┬──────────────────────────────────┬──────────────────────────┬──────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  #  │          Vulnerability           │           File           │                                                   Risk                                                   │
├─────┼──────────────────────────────────┼──────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ 1   │ JWT token in WebSocket URL query │ websocket.service.ts:28  │ Token leaks to browser history, server/proxy logs, Referer headers. Any intermediary can steal the session.                             │
│     │ param                            │                          │                                                                                                          │
├─────┼──────────────────────────────────┼──────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ 2   │ SVG injection — no sanitization  │ icon.ts:8-14, 37, 43, 64 │ cleanSvg() only replaces width/height/color — does NOT strip <script>, event handlers, <iframe>. A compromised SVG at /assets/icons/*.svg executes arbitrary JS via bypassSecurityTrustHtml. │
│     │ on fetched icons                 │                          │                                                                                                          │
└─────┴──────────────────────────────────┴──────────────────────────┴──────────────────────────────────────────────────────────────────────────────────────────────────────────┘

HIGH

┌─────┬──────────────────────────────────┬──────────────────────────┬──────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  #  │          Vulnerability           │           File           │                                                   Risk                                                   │
├─────┼──────────────────────────────────┼──────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ 3   │ Tokens in localStorage (XSS-     │ auth.service.ts:28       │ Supabase persistSession: true stores JWT in localStorage. Any XSS can steal it via localStorage.getItem().                               │
│     │ accessible)                      │                          │                                                                                                          │
├─────┼──────────────────────────────────┼──────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ 4   │ Incomplete logout — doesn't      │ auth.service.ts:76-79    │ After logout, WebSocket stays connected, api.sessions retains data, localSessionIds persists.                                           │
│     │ clear WS, session cache, signals │                          │                                                                                                          │
├─────┼──────────────────────────────────┼──────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ 5   │ No CSP / security headers        │ index.html, server.ts    │ No Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, HSTS. Enables clickjacking, MIME sniffing, inline script injection. │
├─────┼──────────────────────────────────┼──────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ 6   │ File upload — client validation  │ doc-panel.ts:29-49       │ Only HTML accept=".pdf,.txt". No file size limit, no MIME type check, no magic byte validation. Attacker can upload arbitrary files.    │
│     │ only                             │                          │                                                                                                          │
├─────┼──────────────────────────────────┼──────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ 7   │ No rate limiting on auth         │ login.ts, register.ts    │ Unlimited login/register attempts. Enables brute force and credential stuffing.                                                          │
├─────┼──────────────────────────────────┼──────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ 8   │ No HTTP 401 interceptor          │ api.service.ts           │ Expired tokens sent without check. Requests fail silently — no retry with refreshed token.                                               │
└─────┴──────────────────────────────────┴──────────────────────────┴──────────────────────────────────────────────────────────────────────────────────────────────────────────┘

MEDIUM
```

#	Vulnerability	File	Risk
9	WebSocket messages not validated at runtime	websocket.service.ts:37-44	JSON.parse → cast to WsIncoming with zero schema validation. Malformed messages can crash handlers or inject data.
10	Puzzle lines JSON parsed without type check	chat-panel.ts:199-205	JSON.parse(msg.sources) assigned to string[] without verifying it IS an array of strings.
11	Error messages rendered unsanitized	chat-panel.ts:175, 185	Backend error strings interpolated into UI. If backend is compromised, HTML/markdown injection possible.
12	Console.error override is global and permanent	code-editor.ts:190-199	Monkey-patches console globally, suppresses errors matching "Canceled" — could hide real issues.
13	Passwords stored in plain class properties	login.ts:18, register.ts:18-19	Not cleared after use. Accessible in DevTools memory inspector while component lives.
14	Production environment has placeholder URLs	environment.prod.ts:3-4	YOUR_RENDER_URL — app breaks if deployed as-is.
15	Console.error logs in production	Multiple files	Stack traces and implementation details visible in browser console.

LOW

#	Vulnerability	File	Risk
16	Icon name not validated — path traversal possible	icon.ts:47	fetch(\/assets/icons/${iconName}.svg`)` — no regex check on icon name.
17	Email domain check is client-side only	register.ts:26	Backend also validates (confirmed), so this is defense-in-depth only.
18	Supabase anon key in source	environment.ts	Anon key is designed to be public, but exposes project URL to attackers.
19	No email verification flow	register.ts	Users can register with any @e-uvt.ro address without proving ownership.

WHAT'S ALREADY SOLID

- XSS via markdown: DOMPurify.sanitize() IS called before bypassSecurityTrustHtml — this is correct
- Monaco editor: Properly sandboxed (no code execution, suggestions disabled)
- Auth guard: Uses await auth.ready() — handles async session loading
- Route protection: All sensitive routes have canActivate: [authGuard]
- CSRF: Bearer token auth (not cookies) makes CSRF largely irrelevant
- Dependencies: No known CVEs in current versions (Angular 21, marked 18, DOMPurify 3.4)

RECOMMENDED FIX PRIORITY

Before deploy (blocks production):
1. Fix #2 (SVG sanitization) — add DOMPurify to icon component
2. Fix #4 (incomplete logout) — clear all state
3. Fix #5 (security headers) — add to server.ts
4. Fix #6 (file upload validation) — size + MIME check
5. Fix #14 (prod environment URLs)

Soon after deploy:
6. Fix #1 (WS token in URL) — requires backend coordination
7. Fix #7 (rate limiting) — client-side throttle + backend enforcement
8. Fix #8 (401 interceptor) — handle token expiry gracefully
9. Fix #9 (WS message validation) — runtime type checking
10. Fix #15 (production console logs) — conditional on environment.production

Nice to have:
11. Fix #3 (localStorage tokens) — requires Supabase cookie config + backend changes
12. Fix #10-13 (various hardening)

Prompt 12

Question

you are a Angular expert, and I would like you to do an in depth analysis of the entire FE code from this project, looking at every file to analyze the quality of the code, if it follows the all the best design principles, implementation rules, if the code is optimized, if it has any unused/redundant components, if you see repetitive components to create reusable components (html with tailwind, css, ts), if the methods are optimized

Output

I'll do a comprehensive fresh pass focused on reusable patterns, method optimization, and repeated HTML/Tailwind/CSS. Let me read every file systematically.

Here's the complete synthesized audit across all 82 files.

Full Frontend Audit — Post-Refactor (Round 3)

DEAD CODE

Item	File	Line
AuthResponse interface	interfaces.ts	1-6
User interface	interfaces.ts	8-14

Both are defined but never imported anywhere in the codebase.

REUSABLE COMPONENT OPPORTUNITIES (biggest wins)

1. Auth form field — 6 identical instances, ~140 lines

Login, register, and forgot-password all repeat the same field wrapper: label + icon + input + focus ring + optional password toggle. An <app-auth-field> component with inputs for label, icon, type, placeholder, focused, value would collapse 6 field blocks into 6 one-liners.

2. Dropdown pattern — 3 independent implementations, ~110 lines

code-editor, practice, and llm-provider-pill all independently implement: signal toggle + absolute popover + @HostListener('document:click') close + caret-down rotation. Same HTML structure, same behavior. A shared <app-dropdown> (or CDK overlay directive, since CDK is already a dependency) would consolidate all three.

3. Icon button — 14 occurrences across 8 files

The pattern w-9 h-9 rounded-xl grid place-items-center border-none bg-transparent text-(--text-2) transition-[background,color] duration-150 hover:bg-(--sidebar-hover) repeats with minor variations. Fix: a global Tailwind utility class .icon-btn via @apply in styles.css.

4. Empty state block — 3 identical instances in practice.html, ~21 lines

Same icon-in-box + title + subtitle structure. An <app-empty-state icon title subtitle> component.

5. Quick-action cards — 4 identical blocks in chat-panel.html, ~50 lines

Same card with icon + title + subtitle, only data differs. Should be @for over an array.

6. Split/Chat toggle — exact duplicate in top-bar.html, ~25 lines

Lines 20–44 (mobile) and 76–101 (desktop) are copy-pasted. Extract to an ng-template with ngTemplateOutlet, or an EditorToggleComponent.

7. Documents drawer — duplicate in workspace.html + practice.html

Both pages independently render <app-drawer> + <app-doc-panel> and independently call api.listCurriculum() on init. Could move into sidebar-layout itself.

8. Auth footer pill — 3 identical instances, ~12 lines

Same with text + link button in login, register, forgot-password.

9. Auth submit button — 3 identical instances, ~24 lines

Same loading button with opacity-80 + disabled state.

10. Error/success banners — 4 instances across auth forms

Same py-3 px-3.5 rounded-xl alert box pattern (rose for errors, emerald for success).

Total duplicated HTML quantified: ~400+ lines reducible to ~150 lines.

METHOD OPTIMIZATION

11. ChatMessage factory — 5 construction sites in chat-panel.ts

handleWsMessage creates nearly identical ChatMessage objects in 4 switch cases, plus sendWithContent. Each sets 7 null fields identically. Extract:

```
private createMessage(role: 'student'|'tutor', content: string, overrides: Partial<ChatMessage> = {}): ChatMessage {
  return { id: crypto.randomUUID(), session_id: this.sessionId(), role, content,
    mode: null, llm_provider: null, llm_model: null, latency_ms: null,
    sources: null, feedback: null, created_at: new Date().toISOString(), ...overrides };
}
```

Eliminates ~60 lines of boilerplate.

12. renderMarkdown + bypassSecurityTrustHtml — duplicated in 4 components

chat-panel.ts, chat-message.ts, practice.ts, exercise-card.ts all do this.sanitizer.bypassSecurityTrustHtml(renderMarkdown(text)). Extract a shared Angular Pipe.

13. scrollToBottom double-setTimeout in chat-panel.ts

The constructor effect does setTimeout(() => this.scrollToBottom(), 50) and scrollToBottom itself does setTimeout(() => ..., 0). Nested double-timeout. Use a single requestAnimationFrame.

14. loadHistory() called twice on init in workspace.ts

ngOnInit calls loadHistory() directly (line 58), then the paramMap subscription can trigger createNewSession() which calls loadHistory() again (line 241). Remove the direct call in ngOnInit.

TYPE SAFETY

15. WsIncoming is a kitchen-sink interface

All fields optional except type. A discriminated union would be much safer:

```
type WsIncoming =
  | { type: 'response'; content: string; message_id: string; mode: string; sources?: string }
  | { type: 'token'; content: string }
  | { type: 'puzzle'; lines: string[]; content?: string }
  // ...
```

16. ChatMessage.mode is string | null

Should be a union literal: 'socratic' | 'theory' | 'puzzle' | 'puzzle_result' | 'error' | 'code_police_fallback' | null.

17. theme.service.ts — no localStorage validation

localStorage.getItem('theme') as Theme | null — if corrupted (e.g., "blue"), passes through unchecked.

BUGS / EDGE CASES

18. Parsons puzzle stale input

parsons-puzzle.ts copies lines() in ngOnInit only. If the parent changes the lines input (new puzzle on same component instance), the component won't update. Should use effect() to watch for changes.

19. Fragile puzzle-result detection

chat-panel.ts:247 — msg.content.toLowerCase().includes('correct') to detect puzzle success. "Not quite correct" or "Incorrect" would false-positive. Should use a structured is_correct flag from the message.

20. Icon fetch error silently cached

icon.ts:47 — if fetch returns r.ok === false, it caches '' forever. That icon will never be retried. Should not cache failures.

21. Monaco loader onerror unhandled

code-editor.ts:126–128 — script.onload = onLoad but no script.onerror. If vs/loader.js fails to load, user sees an empty editor with no feedback.

22. doc-panel.ts manual alive flag

Uses private alive = true + ngOnDestroy pattern instead of DestroyRef + takeUntilDestroyed (which is already used in workspace.ts). Inconsistent.

MINOR POLISH

23. app.ts — _theme assigned but never read

private _theme = inject(ThemeService) — intent is to force ThemeService initialization. Cleaner: bare inject(ThemeService) without assignment.

24. auth.service.ts — use HttpErrorResponse

The register() catch block uses structural casting. Should check err instanceof HttpErrorResponse for proper HttpClient error handling.

25. cursor-pointer redundancy

styles.css:110 sets button { cursor: pointer } globally, yet cursor-pointer Tailwind class is added on ~20+ buttons. One or the other.

26. truncate shorthand

overflow-hidden text-ellipsis whitespace-nowrap appears 12 times. Tailwind v4 has truncate as a single-class shorthand.

27. Practice tab buttons could be @for loop

3 near-identical tab buttons (~33 lines) could be driven by a config array.

28. app.css is empty

File exists but has no content. Either remove file + styleUrl reference, or keep as placeholder.

WHAT'S DONE WELL

- Signals everywhere — excellent adoption of Angular's signal-based reactivity for state management
- Standalone components — all components properly use imports: [...], no NgModule overhead
- Separate .ts/.html/.css — consistent file convention across all components
- Tailwind adoption — 95%+ of styling is now Tailwind, with CSS files reduced to just :host display rules + framework-required styles (CDK drag, Monaco deep selectors)
- WebSocket service — clean generation-counter pattern to prevent stale reconnects
- Icon component — smart global SVG cache with pending-request deduplication
- Auth guard — properly async with await auth.ready()
- SSR-awareness — isPlatformBrowser checks where needed (theme, icon, Monaco)
- Component inputs/outputs — consistent use of modern input() / output() function API
- Design token system — well-structured CSS custom properties with proper light/dark mode
- Theme persistence — localStorage + system preference fallback

---
WHAT'S SOLID

Area	Status
Zero dead components	None found
Zero constructor DI	100% inject()
Zero inline style="" attributes	Using only Tailwind
Signals & reactivity	Consistent use of signal(), computed(), input(), output()
File conventions	Every component has separate .ts/.html/.css
SSR safety	isPlatformBrowser in theme, icon, code-editor
WebSocket	Generation-counter pattern, proper cleanup
Auth guard	Async with await ready()
Route params	Observable subscription with takeUntilDestroyed
Error handling	All major subscriptions have error callbacks
Design tokens	Clean CSS custom properties, light/dark mode

PRIORITY RANKING

#	Issue	Type	Impact	Effort
1	Auth form field component	Reusable component	-140 lines	Medium
2	Dropdown component	Reusable component	-110 lines	Medium
3	ChatMessage factory method	Method optimization	-60 lines	Low
4	Quick-action cards → @for	Template DRY	-35 lines	Low
5	Markdown pipe	Shared utility	-20 lines, 4 files	Low
6	Icon button utility class	Tailwind utility	14 occurrences	Low
7	Split/Chat toggle extract	Template DRY	-25 lines	Low
8	Empty state component	Reusable component	-15 lines	Low
9	WsIncoming discriminated union	Type safety	Correctness	Low
10	Parsons puzzle stale input	Bug fix	Correctness	Low
11	Dead interfaces removal	Dead code	Cleanup	Trivial
12	truncate shorthand	Polish	12 replacements	Trivial

Prompt 13

Question

I got an email from supabase saying this
Supabase
These issues require
your immediate attention

We detected security vulnerabilities in the projects listed below that could expose your data to unauthorized access. Review and fix them before your data is compromised.

Issues as of 12 Jun 2026:

Critical issue

Table publicly accessible

Anyone with your project URL can read, edit, and delete all data in this table because Row-Level Security is not enabled.

rls_disabled_in_public

Sensitive data publicly accessible

A table with columns that likely contain sensitive data (like passwords or personal identifiers) is accessible through the API without any access restrictions.

sensitive_columns_exposed

```
  Project
  AiSocraticTeachingAssistant
```

Output

```
This is a serious security issue — your Supabase tables are publicly accessible via the API without RLS. Let me investigate your database schema to understand which tables need RLS policies and what
policies to create.

The issue is clear: all 9 tables in your public schema have no RLS policies, so Supabase's PostgREST API exposes them to anyone with your project URL and anon key. Since your app routes all data
access through the FastAPI backend (which connects via DATABASE_URL as a superuser that bypasses RLS), the fix is straightforward: enable RLS on every table with no permissive policies — this blocks
the public API while your backend continues to work normally.
```

Question

```
now all the tables appear in the info tab with this message. is it okay to leave them like this?
```

Output

```
Yes, that's exactly what we want. Here's why:

No policies = no access through the Supabase API. When RLS is enabled with zero policies, the default is deny-all for the anon and authenticated roles. Your data is locked down.

Your FastAPI backend connects via the direct Postgres connection string (DATABASE_URL) as a superuser, which bypasses RLS entirely — so it's unaffected.

The only reason to add policies would be if your frontend queried Supabase tables directly (e.g., supabase.from('users').select()), but it doesn't — it only uses Supabase for auth, and all data goes
through your FastAPI API.

Supabase flags it as informational because most projects use the Supabase client SDK to query tables and would need policies. In your architecture, it's the correct setup.
```

Question

```
and also I have this warning
  Leaked Password Protection Disabled
  security

  Entity
  Auth
  Issue
  Supabase Auth prevents the use of compromised passwords by checking against HaveIBeenPwned.org. Enable this feature to enhance security.

  Description
  Leaked password protection is currently disabled.

  but when I look in supabase settings
  Prevent use of leaked passwords

  Rejects the use of known or easy to guess passwords on sign up or password change. Powered by the HaveIBeenPwned.org Pwned Passwords API. Learn more Only available on Pro plan and above.

  so do I have access to this feature?
```

Output

```
No, you don't — that feature requires the Pro plan. You're on the free tier, so the toggle is locked.

You can safely ignore that warning. Your app already mitigates weak passwords with the 8-character minimum validation in the backend (schemas.py). If you want to go further without upgrading
Supabase, you could add a client-side check against common password lists, but that's optional — the warning itself isn't actionable on the free plan.
```

Prompt 14

Question

```
if I want to delete a user from the users table, and all the entries in all table related to that user, how can I do this? via sql command?
```

Output

```
If your foreign keys have ON DELETE CASCADE, you just delete the user and everything else is removed automatically:

DELETE FROM auth.users WHERE email = 'someone@e-uvt.ro';

Supabase's auth.users is the source of truth — if your public.users table references it with a cascade FK, and your other tables (tutor_sessions, chat_messages, etc.) cascade from public.users, one
delete handles everything.

If you're not sure cascades are set up, you can check with:

SELECT
  tc.table_name, kcu.column_name,
  ccu.table_name AS foreign_table,
  rc.delete_rule
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
JOIN information_schema.referential_constraints rc ON tc.constraint_name = rc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY' AND ccu.table_name = 'users';

If any show NO ACTION or RESTRICT instead of CASCADE, you'd need to either alter the FKs or manually delete in order (children first):

DELETE FROM feedback_events WHERE session_id IN (SELECT id FROM tutor_sessions WHERE user_id = 'THE_USER_UUID');
DELETE FROM chat_messages WHERE session_id IN (SELECT id FROM tutor_sessions WHERE user_id = 'THE_USER_UUID');
DELETE FROM code_snapshots WHERE session_id IN (SELECT id FROM tutor_sessions WHERE user_id = 'THE_USER_UUID');
DELETE FROM parsons_puzzle_events WHERE session_id IN (SELECT id FROM tutor_sessions WHERE user_id = 'THE_USER_UUID');
DELETE FROM research_events WHERE user_id = 'THE_USER_UUID';
DELETE FROM curriculum_documents WHERE user_id = 'THE_USER_UUID';
DELETE FROM tutor_sessions WHERE user_id = 'THE_USER_UUID';
DELETE FROM users WHERE id = 'THE_USER_UUID';
DELETE FROM auth.users WHERE id = 'THE_USER_UUID';

Run the cascade check first — it'll tell you which approach you need.
```

Question

```
i see no action for all the results. what does it imply to make them cascade?
```

Output

```
NO ACTION means the DB will block a delete if child rows exist. Switching to CASCADE means deleting a user automatically deletes all their sessions, messages, etc.

You only need to change the FK constraints in the database — no backend code changes needed.
```

Question

```
can I change the FK constraints in the database directly from supabase interface?
```

Output

```
Yes — in the Supabase dashboard:

1. Go to Table Editor
2. Click a table (e.g. tutor_sessions)
3. Click the FK column (e.g. user_id)
4. Edit the foreign key relationship
5. Change the On Delete action from No action to Cascade
6. Save

Repeat for every FK that references users or chains down (e.g. chat_messages.session_id → tutor_sessions.id).

Just keep in mind — if you change it in the dashboard, your Alembic migrations won't know about it. So if you ever recreate the DB from migrations, the cascades won't be there. You'd want to create
a matching Alembic migration afterward to keep them in sync.
```

Question

```
I made the migration, can I delete a user now?
```

Output

```
All 12 foreign keys now have ON DELETE CASCADE. Deleting a user will automatically clean up all their sessions, messages, code snapshots, feedback, puzzle events, curriculum documents, and research
events. You can now delete a user with just:

DELETE FROM auth.users WHERE email = 'someone@e-uvt.ro';
```